UNIVERSIDAD DEL VALLE DE GUATEMALA

PROGRAMACION ORIENTADA A OBJETOS

DIEGO ISAIAS MARROQUIN MOSCOSO

LABORATORIO #3 - HERENCIA

Análisis

TecniHogar: análisis y diseño de un sistema de ordenes y servicios para la empresa.

REQUISITOS FUNCIONALES DEL SISTEMA

- Cada orden de servicio tiene que contar con un código único por orden, nombre del cliente, dirección, horas estimadas de trabajo y zona metropolitana o periférica.
- El personal también necesita saber en qué estado se encuentra cada orden y cuántas visitas se han realizado.
- La clase principal OrdenServicio debe ser abstracta para evitar crear objetos directamente con ella.
- Las categorías Diagnostico, Instalacion y MantenimientoAireAcondicionado deben heredar de la clase principal.
- InstalacionElectrica debe heredar de la clase Instalacion.
- Diagnóstico no puede superar las 1,5 horas y cuesta Q150 en zona metropolitana o Q225 en periférica, sin importar las horas estimadas.
- Instalación debe tener un atributo llamado puntos a instalar, que debe ser un entero positivo.
- El costo de instalación parte de una tarifa base de Q100 para la zona metropolitana o Q175 para la periférica, más Q90 por cada hora facturable o fracción.
- Las horas facturables se calculan tomando el mayor valor entre las horas estimadas y la cantidad de puntos multiplicada por 0,75. El resultado se redondea hacia arriba.
- Instalación eléctrica requiere además un certificado y una garantía de los materiales.
- A instalación eléctrica se le suman Q120 por el certificado y una garantía equivalente al 5 % del valor declarado de los materiales.
- Mantenimiento de aire acondicionado debe registrar la capacidad del equipo en BTU, que debe estar entre 9 000 y 60 000.

- El mantenimiento de aire acondicionado solo puede realizarse en la zona metropolitana y cuesta la tarifa base de la zona, más Q110 por cada hora estimada y Q80 por gas refrigerante.
- El sistema debe calcular el tiempo estimado de atención según la categoría y la zona.
- Diagnóstico tiene un tiempo de atención de 1 día en zona metropolitana y 2 días en periférica.
- Instalación tiene un tiempo de atención de 3 días en zona metropolitana y 5 días en periférica.
- Instalación eléctrica tiene un tiempo de atención de 2 días adicionales respecto a una instalación normal.
- Mantenimiento de aire acondicionado tiene un tiempo de atención de 2 días.
- El sistema debe permitir cotizar servicios sin registrar órdenes ni modificar los ingresos.
- Cuando un cliente acepta una cotización, se registra la orden, se cobra el monto completo y queda en estado pendiente.
- Cada cuadrilla debe tener un código único, capacidad máxima de horas por jornada y un indicador de certificación en refrigeración.
- El sistema debe permitir crear jornadas y asignar órdenes pendientes a una cuadrilla.
- La suma de horas estimadas de las órdenes asignadas no puede superar la capacidad de la cuadrilla.
- Los mantenimientos de aire acondicionado solo pueden asignarse a cuadrillas certificadas en refrigeración.
- Una orden no puede estar asignada a dos jornadas al mismo tiempo.
- Una cuadrilla no puede tener más de una jornada sin cerrar.

- Mientras una jornada está en preparación, se pueden retirar órdenes y estas vuelven a quedar pendientes.
- Una jornada no puede iniciar si no tiene órdenes asignadas.
- Al iniciar una jornada, sus órdenes pasan al estado en atención.
- Al finalizar una visita, se debe registrar si fue completada o fallida.
- Diagnóstico e instalación permiten hasta 3 visitas, instalación eléctrica hasta 2 y mantenimiento de aire acondicionado solamente 1.
- Si una visita falla y todavía quedan intentos disponibles, la orden vuelve a estar pendiente.
- Si una orden agota sus visitas permitidas, queda cancelada.
- Cada visita adicional a la primera genera un cargo de Q40, que se registra cuando inicia la jornada.
- Una jornada se cierra cuando todas sus órdenes tienen un resultado registrado.
- El sistema debe generar reportes de órdenes por categoría y estado, ingresos por registro, ingresos por visitas adicionales e ingresos por categoría.
- También debe mostrar las jornadas sin cerrar, su cuadrilla, horas asignadas y capacidad disponible.
- Las operaciones inválidas deben rechazarse sin modificar los datos ni los ingresos del sistema.

DECICIONES QUE DEPENDEN DE LA CATEGORIA SERVICIO

- La clase Diagnostico debe encargarse de validar que las horas estimadas no superen 1,5 y calcular su costo según la zona.
- La clase Instalacion debe validar que los puntos a instalar sean positivos y calcular el costo utilizando las horas facturables.
- La clase InstalacionElectrica debe utilizar el cálculo de instalación y agregar el costo del certificado y la garantía de materiales.
- La clase MantenimientoAireAcondicionado debe validar la capacidad en BTU, la zona permitida y calcular el costo del mantenimiento.
- Cada categoría debe determinar sus días estimados de atención, ya que estos cambian dependiendo del servicio y la zona.
- Cada categoría debe determinar su número máximo de visitas permitidas.
- Cada categoría debe determinar si una cuadrilla puede realizar el servicio. En el caso de mantenimiento de aire acondicionado, debe comprobar que tenga certificación en refrigeración.
- Estas decisiones deben estar en las clases correspondientes porque cada tipo de servicio tiene sus propias reglas. Esto permite utilizar herencia y polimorfismo sin tener que comprobar el tipo de servicio desde la clase principal.
- Las validaciones se realizarán tanto en la clase Main como en las clases correspondientes.

ESTADOS DE ORDENES Y JORNADAS

Estados de una orden:

Pendiente: cuando se registra una orden o cuando una visita falla y todavía tiene intentos disponibles.

En atención: cuando inicia la jornada a la que está asignada la orden.

Completada: cuando el servicio se realiza exitosamente.

Cancelada: cuando se alcanza el máximo de visitas permitidas sin completar el servicio.

Una orden puede pasar de pendiente a en atención cuando inicia su jornada. Si la visita se completa, pasa a completada. Si falla, vuelve a pendiente siempre que tenga visitas disponibles; de lo contrario, pasa a cancelada.

Estados de una jornada:

En preparación: cuando se crea la jornada y se pueden asignar o retirar órdenes.

En curso: cuando inicia la jornada y sus órdenes pasan a estar en atención.

Cerrada: cuando se registran los resultados de todas las órdenes asignadas.

Una jornada pasa de preparación a en curso cuando se inicia, siempre que tenga órdenes asignadas. Después pasa a cerrada cuando se registran todos los resultados. Al cerrarse, la cuadrilla queda disponible para otra jornada.

CLASES NECESARIAS Y SUS RESPONSABILIDADES

Clase OrdenServicio (abstracta)

Es la clase principal que contiene la información común de todas las órdenes de servicio. Debe ser abstracta para evitar crear objetos directamente de ella y permitir que las clases hijas implementen sus propias reglas.

Atributos “protected”:

- # codigo: String: identificador único de la orden.
- # nombreCliente: String: nombre del cliente.
- # direccion: String: dirección donde se realizará el servicio.
- # horasEstimadas: double: horas estimadas de trabajo.
- # zona: String: zona metropolitana o periférica.
- # estado: String: estado actual de la orden.
- # visitasRealizadas: int: cantidad de visitas realizadas.
- # asignada: boolean: indica si la orden está asignada actualmente a una jornada.
- # cobrada: boolean: indica si el costo inicial de la orden ya fue cobrado.

Atributo privado:

- ingresosAdicionales: double: total acumulado por visitas adicionales de la orden.

Métodos públicos:

- + OrdenServicio(codigo: String, nombreCliente: String, direccion: String, horasEstimadas: double, zona: String)
- + calcularCosto(): double: método abstracto para calcular el costo del servicio.
- + calcularDiasAtencion(): int: método abstracto para obtener los días estimados de atención.
- + obtenerMaximoVisitas(): int: método abstracto para determinar el número máximo de visitas.
- + getCategoria(): String: método abstracto que devuelve el nombre de la categoría usado en los reportes.
- + esCompatible(cuadrilla: Cuadrilla): boolean: determina si la cuadrilla puede realizar el servicio.
- + iniciarAtencion(): boolean: cambia una orden Pendiente al estado En atención y devuelve false si no puede iniciarse.
- + registrarResultadoVisita(completada: boolean): boolean: incrementa las visitas realizadas y cambia el estado a Completada, Pendiente o Cancelada según el resultado y el máximo permitido.
- + getCodigo(): String: devuelve el código de la orden.
- + getAsignada(): boolean: devuelve true si la orden está asignada a una jornada.
- + getNombreCliente(): String: devuelve el nombre del cliente de la orden.
- + getIngresosAdicionales(): double: devuelve los cobros acumulados por visitas adicionales.
- + getDireccion(): String: devuelve la dirección del servicio.

- + getHorasEstimadas(): double: devuelve las horas estimadas.
- + getZona(): String: devuelve la zona.
- + getEstado(): String: devuelve el estado actual.
- + getVisitasRealizadas(): int: devuelve la cantidad de visitas realizadas.
- + isCobrada(): boolean: indica si el costo inicial ya se registró como ingreso.
- + setAsignada(asignada: boolean): void: actualiza el indicador que controla si la orden pertenece a una jornada.
- + setCobrada(cobrada: boolean): void: actualiza el indicador de cobro inicial.
- + agregarIngresoAdicional(cantidad: double): void: valida y acumula un ingreso positivo por una visita adicional.
- + toString(): String: devuelve una representación de la orden con sus datos comunes, estado y visitas realizadas.

Su constructor valida los datos obligatorios, las horas positivas y finitas, y que la zona sea Metropolitana o Periferica. Inicializa la orden en estado Pendiente, con cero visitas, sin asignación, sin cobro y sin ingresos adicionales.

Clase Diagnostico

Hereda de OrdenServicio y se encarga de calcular el costo y validar las restricciones de un diagnóstico.

Métodos públicos:

- + Diagnostico(codigo: String, nombreCliente: String, direccion: String, horasEstimadas: double, zona: String)
- + calcularCosto(): double: devuelve Q150 en zona metropolitana o Q225 en periférica.
- + calcularDiasAtencion(): int: devuelve 1 día en zona metropolitana o 2 en periférica.
- + obtenerMaximoVisitas(): int: devuelve un máximo de 3 visitas.
- + getCategoria(): String: devuelve "Diagnostico" para clasificar la orden en los reportes.
- + toString(): String: sobrescribe el método heredado y agrega una línea separadora al final de los datos comunes del diagnóstico.

Su constructor recibe los datos comunes y utiliza super(...) para inicializarlos. También valida que las horas estimadas no superen 1,5.

Clase Instalacion

Hereda de OrdenServicio y se encarga de calcular las horas facturables y el costo de una instalación.

Atributo privado:

- - puntosInstalar: int: cantidad de puntos que se deben instalar.

Métodos públicos:

- + Instalacion(codigo: String, nombreCliente: String, direccion: String, horasEstimadas: double, zona: String, puntosInstalar: int)
- + calcularHorasFacturables(): double: calcula el mayor valor entre las horas estimadas y los puntos multiplicados por 0,75, redondeando hacia arriba.
- + calcularCosto(): double: calcula la tarifa base más Q90 por cada hora facturable.
- + calcularDiasAtencion(): int: devuelve 3 días en zona metropolitana o 5 en periférica.
- + obtenerMaximoVisitas(): int: devuelve un máximo de 3 visitas.
- + getCategoria(): String: devuelve "Instalacion" para clasificar la orden en los reportes.
- + getPuntosInstalar(): int: devuelve la cantidad de puntos.
- + toString(): String: sobrescribe el método heredado, agrega puntosInstalar a los datos comunes y cierra la representación con una línea separadora.

Su constructor recibe los datos comunes y los puntos a instalar. Utiliza super(...) para inicializar los atributos heredados y valida que los puntos sean un entero positivo.

Clase InstalacionElectrica

Hereda de Instalacion porque utiliza las mismas reglas de una instalación normal, pero agrega el costo del certificado y la garantía de materiales.

Atributo privado:

- - valorMateriales: double: valor declarado de los materiales.

Métodos públicos:

- + InstalacionElectrica(codigo: String, nombreCliente: String, direccion: String, horasEstimadas: double, zona: String, puntosInstalar: int, valorMateriales: double)
- + calcularCosto(): double: utiliza el costo de instalación y agrega Q120 por el certificado y el 5 % del valor de los materiales.
- + calcularDiasAtencion(): int: devuelve 2 días adicionales respecto a una instalación normal.
- + obtenerMaximoVisitas(): int: devuelve un máximo de 2 visitas.
- + getCategoria(): String: devuelve "Instalacion Electrica" para clasificar la orden en los reportes.
- + getValorMateriales(): double: devuelve el valor declarado de los materiales.
- + toString(): String: muestra los datos comunes, los puntos a instalar y el valor de los materiales, y cierra la representación con una línea separadora.

Su constructor utiliza super(...) para inicializar los datos heredados de Instalacion y después valida e inicializa el valor de los materiales.

Clase MantenimientoAireAcondicionado

Hereda de OrdenServicio y se encarga de las reglas relacionadas con el mantenimiento de aire acondicionado.

Atributo privado:

- - capacidadBTU: int: capacidad del equipo en BTU.

Métodos públicos:

- + MantenimientoAireAcondicionado(codigo: String, nombreCliente: String, direccion: String, horasEstimadas: double, zona: String, capacidadBTU: int)
- + calcularCosto(): double: calcula la tarifa base metropolitana, más Q110 por cada hora estimada y Q80 por gas refrigerante.
- + calcularDiasAtencion(): int: devuelve 2 días.
- + obtenerMaximoVisitas(): int: devuelve un máximo de 1 visita.
- + getCategoria(): String: devuelve "Mantenimiento" para clasificar la orden en los reportes.

- + esCompatible(cuadrilla: Cuadrilla): boolean: comprueba que la cuadrilla tenga certificación en refrigeración.
- + getCapacidadBTU(): int: devuelve la capacidad del equipo.
- + toString(): String: sobrescribe el método heredado, agrega capacidadBTU a los datos comunes y cierra la representación con una línea separadora.

Su constructor utiliza super(...) para inicializar los datos comunes y valida que la capacidad esté entre 9 000 y 60 000 BTU. También comprueba que la zona sea metropolitana.

Clase Cuadrilla

Se encarga de almacenar la información de cada cuadrilla y su capacidad de trabajo.

Atributos privados:

- - codigo: String: identificador único de la cuadrilla.
- - capacidadHoras: double: cantidad máxima de horas de trabajo por jornada.
- - certificadaRefrigeracion: boolean: indica si la cuadrilla tiene certificación en refrigeración.

Métodos públicos:

- + Cuadrilla(codigo: String, capacidadHoras: double, certificadaRefrigeracion: boolean)
- + getCodigo(): String: devuelve el código de la cuadrilla.
- + getCapacidadHoras(): double: devuelve su capacidad máxima.
- + isCertificadaRefrigeracion(): boolean: indica si cuenta con certificación.
- + toString(): String: devuelve el código, la capacidad de horas y el indicador de certificación en refrigeración de la cuadrilla.

Su constructor recibe el código, la capacidad máxima de horas y la certificación en refrigeración. Se validará tanto desde Main como desde la clase Cuadrilla que el código no esté vacío y que la capacidad sea positiva. Main comprobará los datos ingresados por el usuario, mientras que el constructor de Cuadrilla verificará que los valores recibidos sean válidos antes de inicializar el objeto.

Clase Jornada

Se encarga de administrar las órdenes asignadas a una cuadrilla y controlar el desarrollo de la jornada.

Atributos privados:

- - codigo: String: identificador de la jornada.
- - cuadrilla: Cuadrilla: cuadrilla encargada.
- - ordenes: List<OrdenServicio>: lista de órdenes asignadas.
- - estado: String: estado actual de la jornada.
- - resultadosRegistrados: List<String>: guarda los códigos de las órdenes cuyo resultado ya fue registrado en la visita actual.

Métodos públicos:

- + Jornada(codigo: String, cuadrilla: Cuadrilla)
- + getEstado(): String: devuelve el estado actual de la jornada.
- + getCodigo(): String: devuelve el código identificador de la jornada.
- + getCuadrilla(): Cuadrilla: devuelve la cuadrilla asignada a la jornada.
- + getOrdenes(): List<OrdenServicio>: devuelve una copia de la lista para impedir que código externo modifique directamente las órdenes asignadas.
- + asignarOrden(orden: OrdenServicio): boolean: agrega una orden pendiente, libre, compatible y que no supere la capacidad; después la marca como asignada.
- + retirarOrden(codigo: String): boolean: retira la orden por código mientras la jornada está en Preparacion y la vuelve a marcar como no asignada.

- + iniciarJornada(): boolean: comprueba que existan órdenes pendientes y asignadas, inicia su atención y cambia el estado a En curso.
- + registrarResultado(codigoOrden: String, completada: boolean): boolean: registra una sola vez el resultado de la orden; cuando todas tienen resultado, cierra la jornada y las libera.
- + calcularHorasAsignadas(): double: suma las horas estimadas de las órdenes incluidas.
- + calcularCapacidadDisponible(): double: resta las horas asignadas de la capacidad de la cuadrilla.
- + estaCerrada(): boolean: devuelve true cuando el estado es Cerrada.
- + toString(): String: muestra el código y estado, la cuadrilla y los códigos de las órdenes asignadas.

Su constructor valida que el código no esté vacío y que la cuadrilla no sea nula. Después inicializa ordenes y resultadosRegistrados como listas vacías, y establece el estado inicial como "Preparacion", igual que en el código.

La asignación mantiene sincronizados la jornada y la orden. asignarOrden(...) cambia asignada a true solamente después de superar todas las validaciones, mientras que retirarOrden(...) y el cierre de la jornada vuelven a establecerla en false.

iniciarJornada() delega en cada OrdenServicio el cambio a En atención. registrarResultado(...) también delega el resultado en la orden, registra su código para evitar duplicados y cierra la jornada únicamente cuando todas las órdenes recibieron un resultado.

La lista resultadosRegistrados guarda los códigos de las órdenes cuyo resultado ya fue procesado. Esto impide registrar dos veces la misma visita. Cuando su tamaño coincide con el número de órdenes, registrarResultado(...) cambia el estado a "Cerrada" y libera todas las órdenes mediante setAsignada(false).

getOrdenes() crea y devuelve una nueva ArrayList con las órdenes actuales. Sistema utiliza esa copia para revisar las visitas previas y registrar los cargos adicionales sin exponer la lista interna de Jornada.

Clase Sistema

Se encarga de administrar las órdenes, cuadrillas, jornadas, ingresos y reportes del sistema.

Atributos privados:

- - ordenes: List<OrdenServicio>: almacena las órdenes registradas.
- - cuadrillas: List<Cuadrilla>: almacena las cuadrillas.
- - jornadas: List<Jornada>: almacena las jornadas.
- - ingresosRegistro: double: ingresos obtenidos al registrar órdenes.
- - ingresosVisitas: double: ingresos por visitas adicionales.

Métodos públicos:

- + Sistema()
- + cotizarServicio(orden: OrdenServicio): double: calcula el costo sin registrar la orden ni modificar ingresos.
- + registrarOrden(orden: OrdenServicio): boolean: valida el estado inicial, evita códigos y cobros duplicados, registra la orden, suma su costo y la marca como cobrada.
- + registrarCuadrilla(cuadrilla: Cuadrilla): boolean: registra una cuadrilla existente cuyo código no esté repetido.
- + crearJornada(codigo: String, cuadrilla: Cuadrilla): boolean: crea una jornada con código único para una cuadrilla registrada que no tenga otra jornada abierta.
- + iniciarJornada(codigoJornada: String): boolean: inicia la jornada y cobra Q40 por cada orden que ya tenga una visita previa.
- + buscarCuadrilla(codigo: String): Cuadrilla: busca una cuadrilla por código o devuelve null.

- + buscarJornada(codigo: String): Jornada: busca una jornada por código o devuelve null.
- + buscarOrden(codigo: String): OrdenServicio: busca una orden por código o devuelve null.
- + mostrarOrdenesPorEstado(categoria: String, estado: String): void: imprime las órdenes que coinciden con la categoría y el estado, junto con su cantidad.
- + generarReporteOrdenes(): void: agrupa e imprime las órdenes por categoría y estado.
- + generarReporteIngresos(): void: muestra ingresos iniciales, visitas adicionales, total general y total por categoría.
- + generarReporteJornadas(): void: muestra jornadas en Preparacion y En curso, sus horas asignadas y la capacidad disponible.
- + agregarOrdenInicial(orden: OrdenServicio): boolean: carga una orden de prueba válida y no cobrada sin aumentar los ingresos.

Su constructor inicializa las listas y los acumuladores de ingresos en cero. registrarOrden(...) acepta solamente órdenes nuevas en estado Pendiente, sin visitas ni asignación y que todavía no estén cobradas; después registra el costo en ingresoRegistro y activa cobrada.

iniciarJornada(...) delega el cambio de estado a Jornada. Si la operación tiene éxito, suma Q40 a ingresoVisitas por cada orden con visitas anteriores y registra el mismo valor en ingresosAdicionales de esa orden. Así, generarReporteIngresos() puede calcular correctamente los totales generales y por categoría.

agregarOrdenInicial(...) se usa para los datos de prueba de Main. Exige una orden Pendiente, sin visitas, asignación ni cobro, pero no modifica ingresoRegistro; por eso esas órdenes iniciales aparecen en las consultas sin contarse como ventas realizadas durante la ejecución.

Clase Main

Se encarga de ejecutar el programa, mostrar el menú y recibir los datos ingresados por el usuario.

Atributos privados:

- - sistema: Sistema: objeto estático que administra órdenes, cuadrillas, jornadas e ingresos.
- - scanner: Scanner: lector estático de las entradas del usuario.

Métodos públicos:

- + main(args: String[]): void: inicializa el programa, carga los datos de prueba y dirige las once opciones del menú.
- + mostrarMenu(): void: imprime las opciones principales de TecniHogar.
- + registrarOrden(): void: solicita, valida y registra una orden de cualquiera de las cuatro categorías.
- + registrarCuadrilla(): void: solicita los datos, valida la certificación y registra una cuadrilla con código único.
- + registrarJornadas(): void: crea una jornada con código único para una cuadrilla registrada y disponible.
- + consultarInformacion(): void: permite buscar y mostrar una orden o una cuadrilla por código.
- + consultarMostrarReportes(): void: presenta el submenú para generar reportes de órdenes, ingresos o jornadas.
- + leerEntero(mensaje: String): int: repite la lectura hasta recibir un entero no negativo y no vacío.

- + leerDouble(mensaje: String): double: repite la lectura hasta recibir un número finito, no negativo y no vacío.
- + leerString(mensaje: String): String: repite la lectura hasta recibir texto no vacío.
- + esperarEnter(): void: pausa la ejecución hasta que el usuario presione Enter.
- + cotizarServicio(): void: crea temporalmente el servicio solicitado, muestra costo y días, pero no lo registra.
- + asignarOrdenJornada(): void: busca la jornada y la orden, e intenta asignarlas respetando estado, compatibilidad y capacidad.
- + retirarOrdenJornada(): void: retira una orden de una jornada que todavía esté en Preparacion.
- + iniciarJornada(): void: valida la jornada y solicita a Sistema que la inicie y registre los cargos adicionales.
- + registrarResultadoVisita(): void: solicita el resultado, lo registra en Jornada y muestra el estado y las visitas de la orden.

main(...) crea Sistema y Scanner, registra tres cuadrillas y carga ocho órdenes iniciales mediante agregarOrdenInicial(...). Después mantiene el menú activo hasta elegir la opción 11 y llama a un método específico para cada operación.

registrarOrden() y cotizarServicio() solicitan los datos comunes y los particulares de cada categoría. Ambos construyen el objeto dentro de un bloque try-catch; la diferencia es que registrarOrden() llama a Sistema.registrarOrden(...) y genera el cobro, mientras que cotizarServicio() solo calcula y muestra el resultado.

Las operaciones de jornada están separadas en registrarJornadas(), asignarOrdenJornada(), retirarOrdenJornada(), iniciarJornada() y registrarResultadoVisita(). Main comprueba la existencia y el estado de los objetos, pero las reglas y cambios definitivos se delegan a Sistema, Jornada y OrdenServicio.

leerEntero(), leerDouble() y leerString() mantienen ciclos de validación para impedir entradas vacías o numéricas inválidas. consultarInformacion() y consultarMostrarReportes() usan submenús independientes, mientras esperarEnter() controla las pausas antes de volver al menú principal.

Validación de zonas y estados

Para simplificar el diseño, las zonas y estados se representarán mediante atributos de tipo String, sin crear clases adicionales.

- Las zonas permitidas serán "Metropolitana" y "Periferica".
- Los estados de una orden serán "Pendiente", "En atención", "Completada" y "Cancelada".
- Los estados de una jornada serán "Preparación", "En curso" y "Cerrada".

Las zonas se validarán tanto al recibir los datos en Main como al construir una orden. Los estados serán modificados únicamente por los métodos responsables de las transiciones, para evitar cambios inválidos.

Justifique por qué la instalación eléctrica certificada se relaciona con la instalación de la forma en que usted propone. Explique qué ganaría o perdería con una alternativa.

La clase InstalacionElectrica hereda de Instalacion porque comparte sus atributos y métodos, como la cantidad de puntos a instalar, el cálculo de horas facturables y la tarifa base según la zona.

La diferencia es que InstalacionElectrica agrega el valor de los materiales, Q120 por el certificado y una garantía equivalente al 5 % del valor de los materiales. También tiene 2 días adicionales de atención y permite un máximo de 2 visitas.

Al utilizar herencia se evita repetir código, ya que InstalacionElectrica puede reutilizar los métodos de Instalacion mediante super y sobrescribir los que necesitan un comportamiento diferente.

Una alternativa sería que InstalacionElectrica heredara directamente de OrdenServicio. Esto permitiría manejar sus reglas de forma independiente, pero sería necesario repetir los atributos y métodos relacionados con la instalación, como los puntos a instalar y el cálculo de horas facturables. Esto aumentaría la cantidad de código y dificultaría su mantenimiento.

Por eso, se considera más conveniente que InstalacionElectrica herede de Instalacion, aprovechando las características que ambas comparten.

Incorporación de nuevas categorías de servicio

Para agregar una nueva categoría, como instalación de paneles solares o cámaras de seguridad, se tendría que crear una nueva clase que herede de OrdenServicio o de Instalacion, dependiendo de las características que comparta con las categorías existentes.

La nueva clase tendría sus propios atributos y debería implementar calcularCosto(), calcularDiasAtencion(), obtenerMaximoVisitas() y getCategoria(). También podría sobrescribir esCompatible() según las reglas de cuadrilla y toString() cuando agregue atributos propios que deban mostrarse.

También sería necesario modificar Main para agregar la opción de cotizar y registrar el nuevo servicio, solicitando los datos específicos que necesite.

La clase Sistema no tendría que modificar su estructura principal, ya que las órdenes se almacenan mediante una lista de tipo OrdenServicio, lo que permite guardar objetos de cualquiera de sus clases hijas.

Los reportes tampoco tendrían que modificarse si utilizan polimorfismo para consultar las órdenes y agruparlas por su categoría mediante getCategoria(). De igual forma, Jornada puede seguir usando los métodos de OrdenServicio para comprobar compatibilidad, horas asignadas y visitas, mientras Sistema coordina los registros y los cobros.

De esta manera, el sistema podría incorporar nuevas categorías sin modificar las reglas de las clases existentes, aprovechando la herencia y el polimorfismo.

PRUEBAS DE FUNCIONAMIENTO

Prueba 1 – Diagnóstico metropolitano

- Datos de entrada: zona Metropolitana, 1,5 horas estimadas.
- Resultado esperado: costo Q150,00 y 1 día de atención.
- Resultado obtenido: ====== RESULTADO DE COTIZACION ======

  Costo del servicio: Q150.00

  Dias estimados de atencion: 1

- Evidencia:

```
1

====== COTIZAR SERVICIO ======
1. Diagnostico
2. Instalacion
3. Instalacion Electrica
4. Mantenimiento de Aire Acondicionado
5. Regresar al menu principal

Seleccione una opcion:
1

Ingrese el codigo de la orden:

prueba

Ingrese el nombre del cliente:

prueba

Ingrese la direccion:

prueba

Ingrese las horas estimadas:
1.5

Ingrese la zona (Metropolitana/Periferica):

Metropolitana

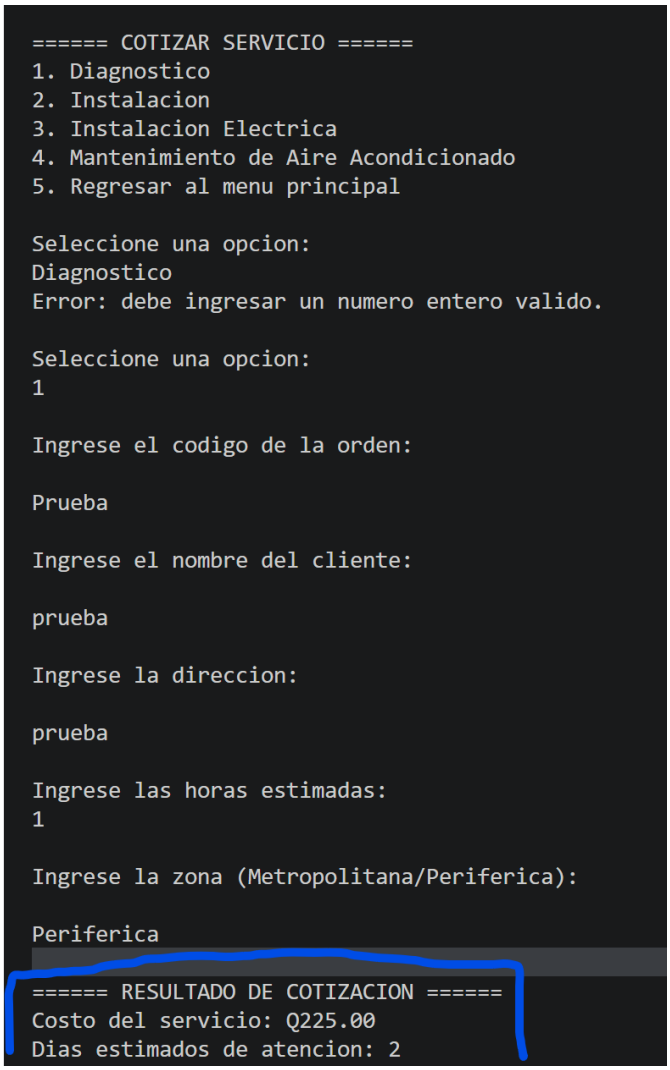
====== RESULTADO DE COTIZACION ======
Costo del servicio: Q150.00
Dias estimados de atencion: 1
```

Prueba 2 – Diagnóstico periférico

- Datos de entrada: zona Periférica, 1 hora estimada.
- Resultado esperado: costo Q225,00 y 2 días de atención.
- Resultado obtenido: ====== RESULTADO DE COTIZACION ======

  Costo del servicio: Q225.00

  Dias estimados de atencion: 2
- Evidencia:



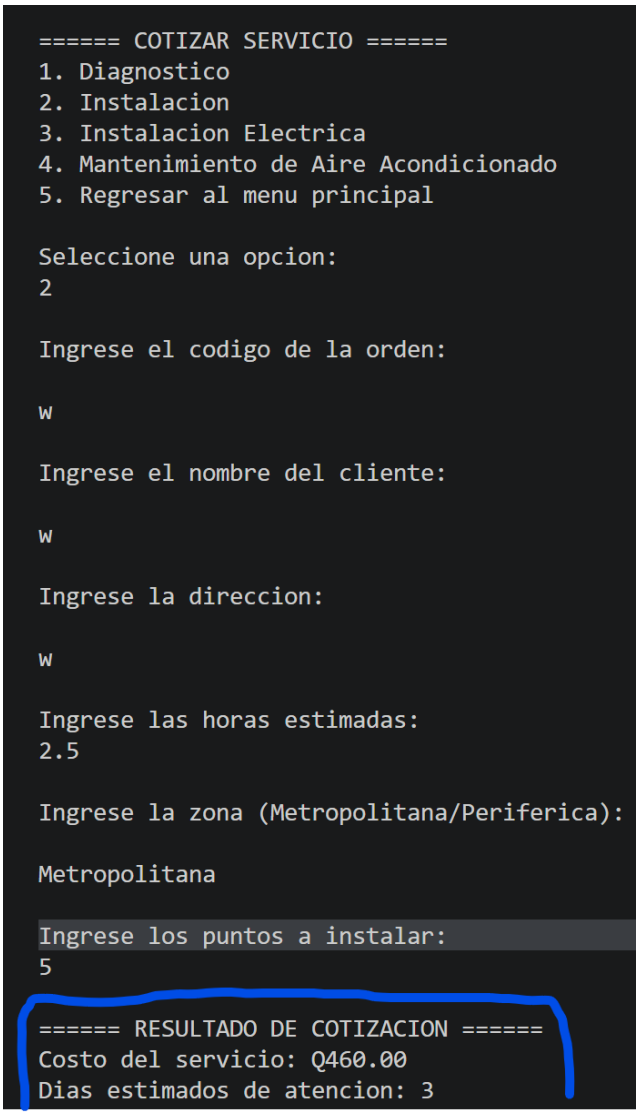
Prueba 3 – Instalación con horas por puntos

- Datos de entrada: zona Metropolitana, 2,5 horas estimadas y 5 puntos a instalar.
- Resultado esperado: costo Q460,00 y 3 días de atención.
- Resultado obtenido: ====== RESULTADO DE COTIZACION ======

  Costo del servicio: Q460.00

  Dias estimados de atencion: 3

- Evidencia:

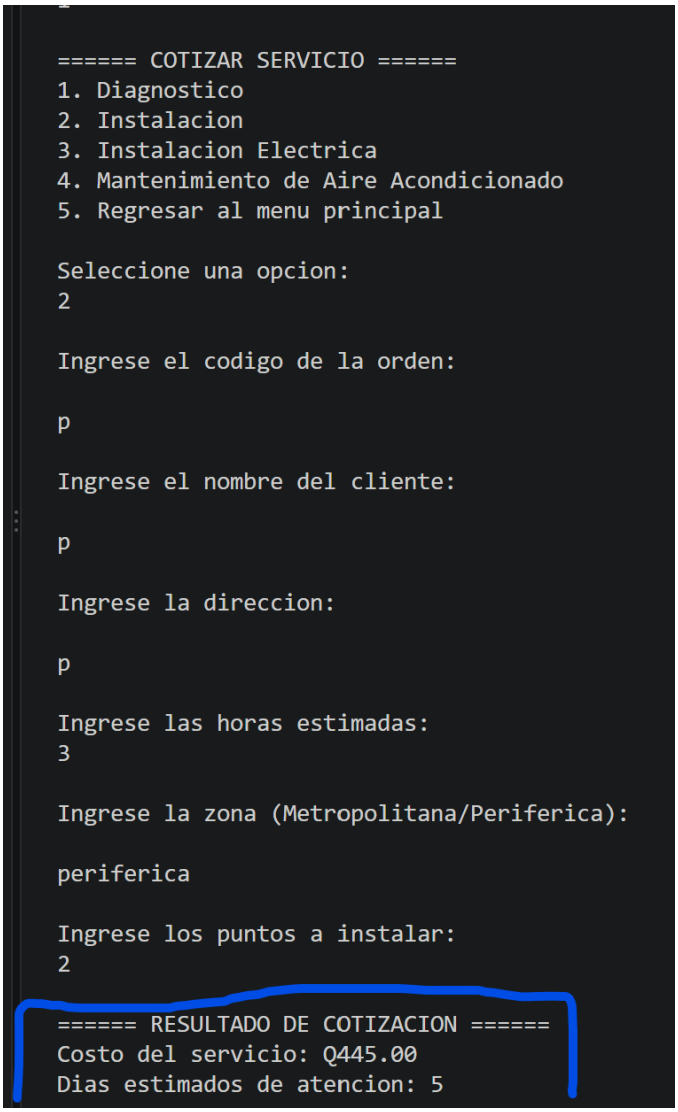


Prueba 4 – Instalación periférica

- Datos de entrada: zona Periférica, 3 horas estimadas y 2 puntos a instalar.
- Resultado esperado: costo Q445,00 y 5 días de atención.
- Resultado obtenido: ======= RESULTADO DE COTIZACION ======

  Costo del servicio: Q445.00

  Dias estimados de atencion: 5
- Evidencia:



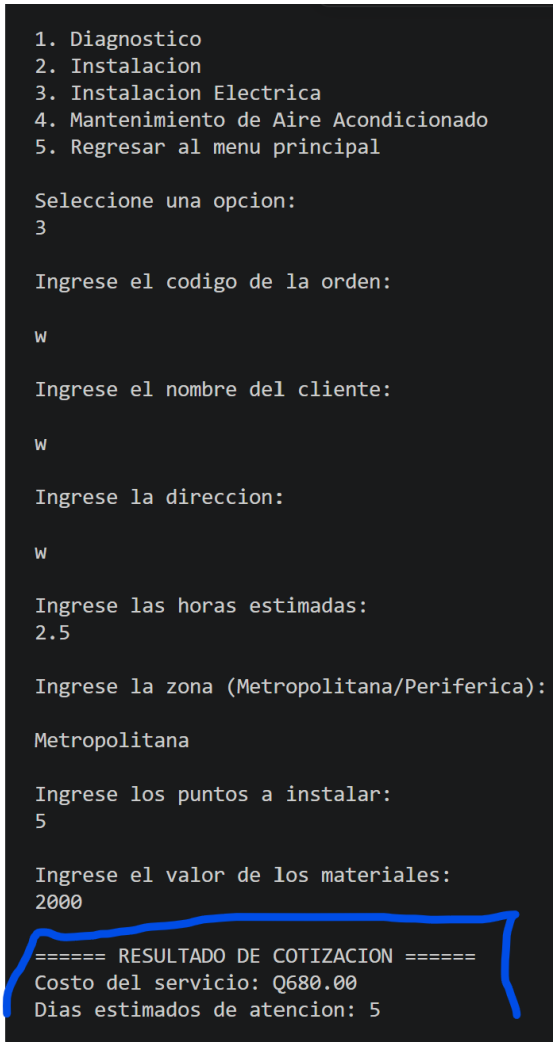
Prueba 5 – Instalación eléctrica certificada

- Datos de entrada: zona Metropolitana, 2,5 horas estimadas, 5 puntos y materiales valorados en Q2 000,00.
- Resultado esperado: costo Q680,00 y 5 días de atención.
- Resultado obtenido: ====== RESULTADO DE COTIZACION ======

  Costo del servicio: Q680.00

  Dias estimados de atencion: 5

- Evidencia:



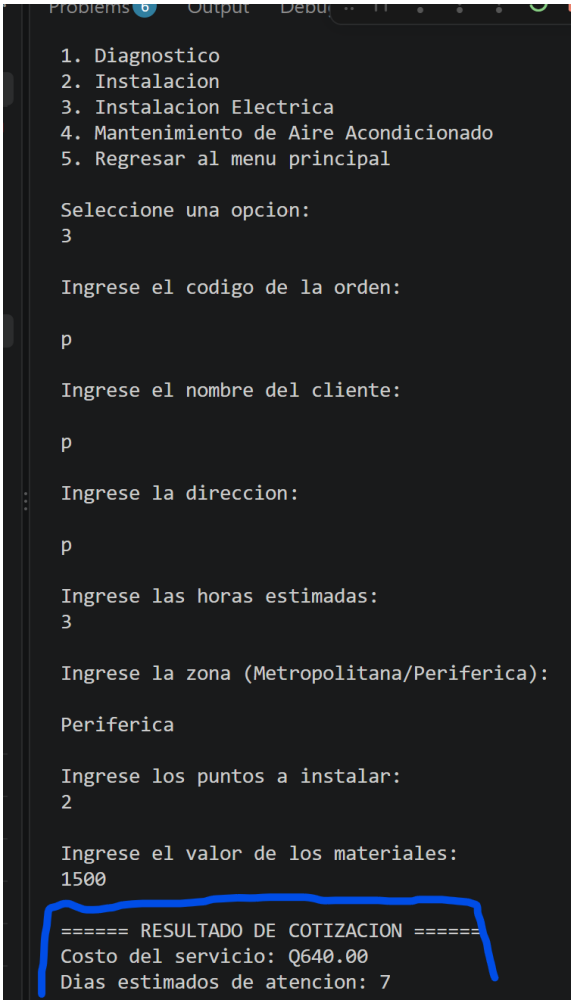
Prueba 6 – Instalación eléctrica periférica

- Datos de entrada: zona Periférica, 3 horas estimadas, 2 puntos y materiales valorados en Q1 500,00.
- Resultado esperado: costo Q640,00 y 7 días de atención.
- Resultado obtenido: ====== RESULTADO DE COTIZACION ======

  Costo del servicio: Q640.00

  Dias estimados de atencion: 7

- Evidencia:

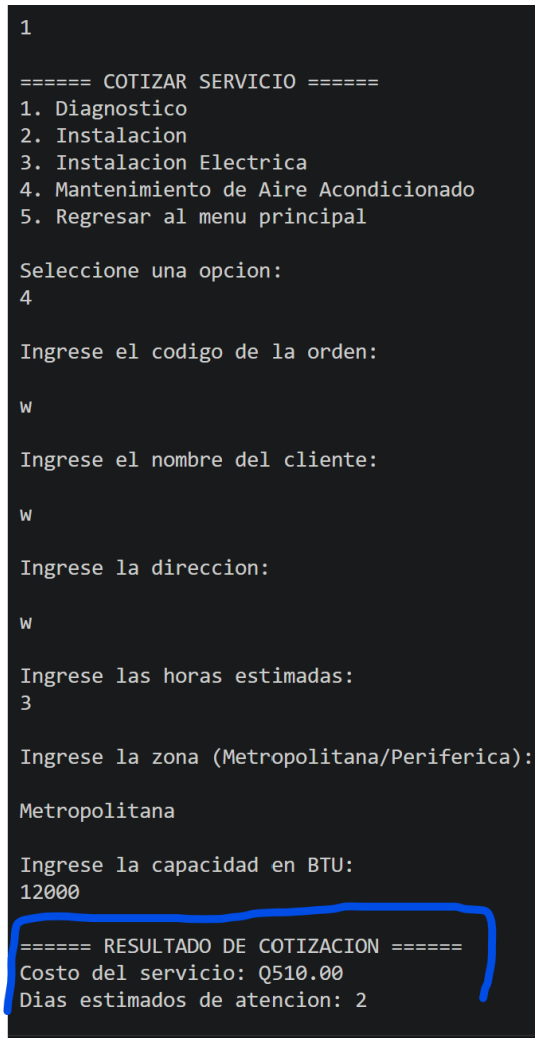


Prueba 7 – Mantenimiento de aire acondicionado

- Datos de entrada: zona Metropolitana, 3 horas estimadas y equipo de 12 000 BTU.
- Resultado esperado: costo Q510,00 y 2 días de atención.
    - Resultado obtenido: ====== RESULTADO DE COTIZACION ======

      Costo del servicio: Q510.00

      Dias estimados de atencion: 2
- Evidencia:



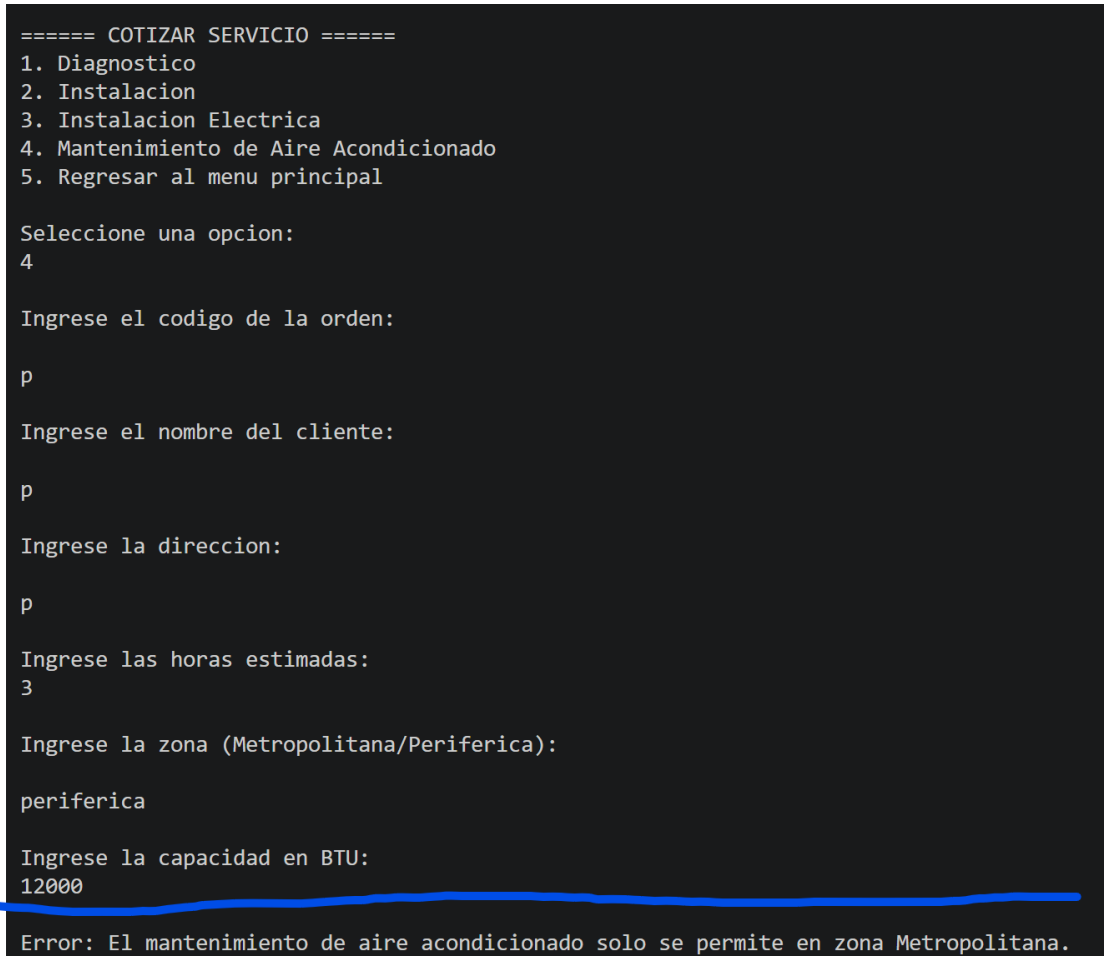
Prueba 8 – Diagnóstico con horas fuera del límite

- Datos de entrada: diagnóstico con 1,6 horas estimadas.
- Resultado esperado: el sistema rechaza la operación porque un diagnóstico no puede superar 1,5 horas.
- Resultado obtenido: Error: Un diagnóstico no puede superar las 1.5 horas.
- Evidencia:



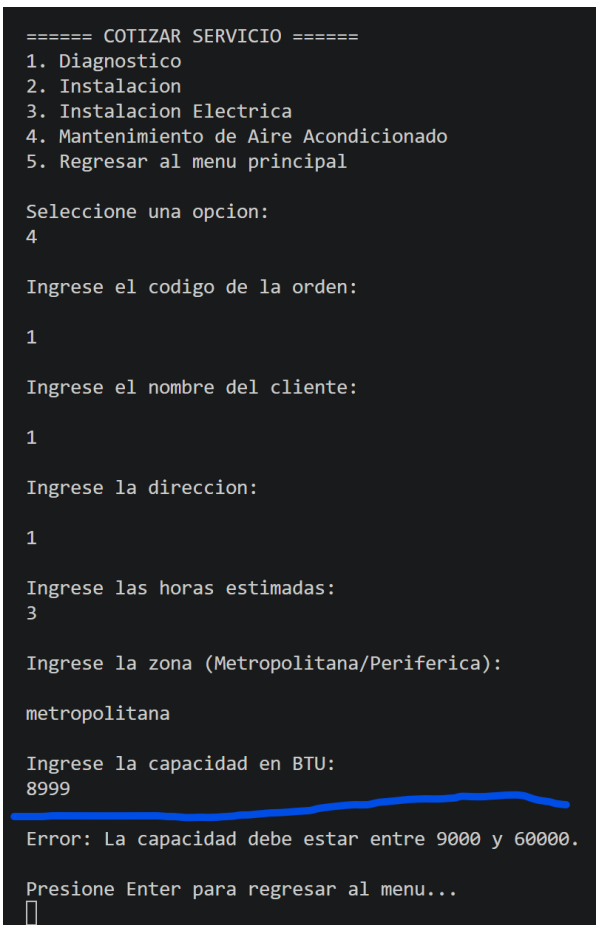
Prueba 9 – Mantenimiento en zona periférica

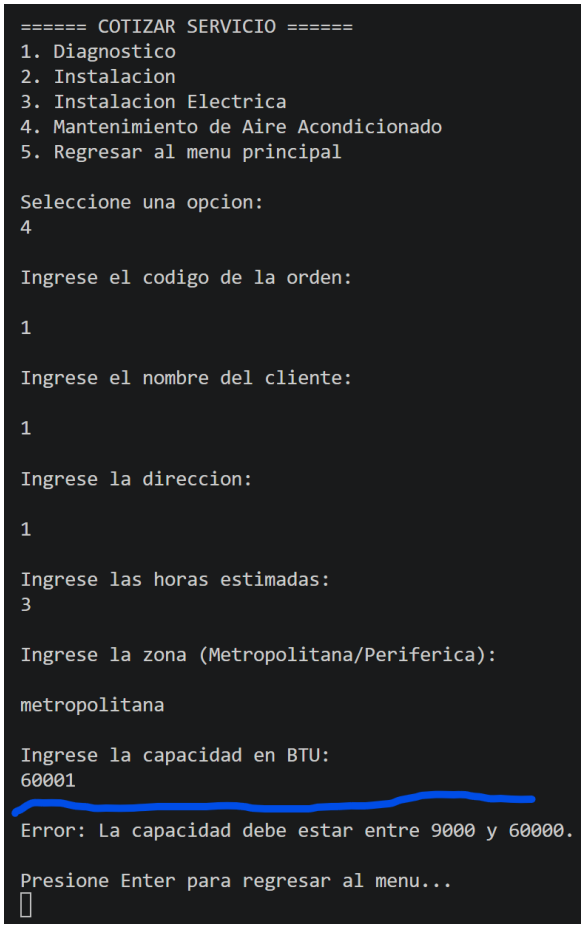
- Datos de entrada: mantenimiento de aire acondicionado en zona Periférica, 12 000 BTU.
- Resultado esperado: el sistema rechaza el servicio porque únicamente se permite en zona Metropolitana.
- Resultado obtenido: El mantenimiento de aire acondicionado solo se permite en zona Metropolitana
- Evidencia:



Prueba 10 – Capacidad BTU fuera del rango

- Datos de entrada: intentar registrar mantenimientos con 8 999 BTU y 60 001 BTU.
- Resultado esperado: el sistema rechaza ambos valores porque la capacidad permitida está entre 9 000 y 60 000 BTU.
- Resultado obtenido: Error: La capacidad debe estar entre 9000 y 60000.
- Evidencia:

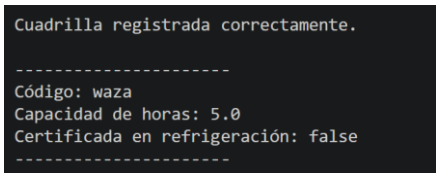




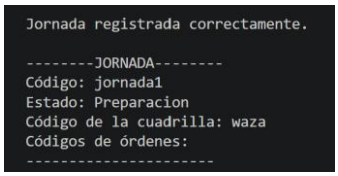
Prueba 11 – Asignación que supera la capacidad

- Datos de entrada: cuadrilla con capacidad de 5 horas; asignar una orden de 3 horas y después otra de 2,5 horas.
- Resultado esperado: la primera orden se asigna correctamente y la segunda se rechaza porque el total sería 5,5 horas.
- Resultado obtenido: Orden registrada correctamente. Y No se pudo asignar la orden.
- Evidencia:

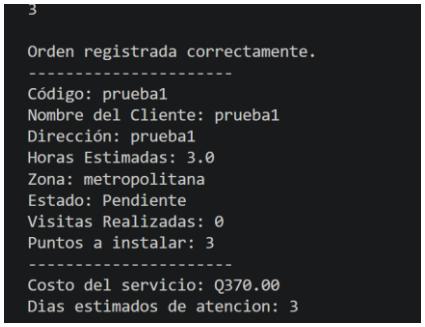
Cuadrilla a usar



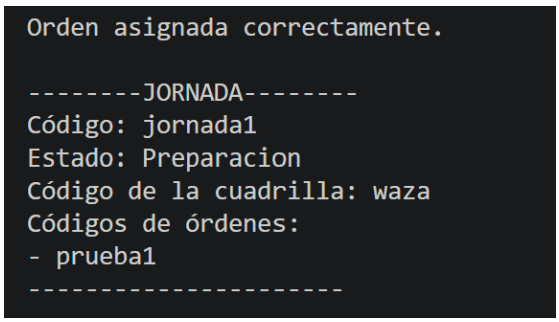
Jornada a usar



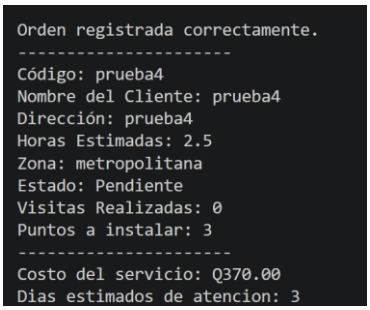
Orden de 3h a usar

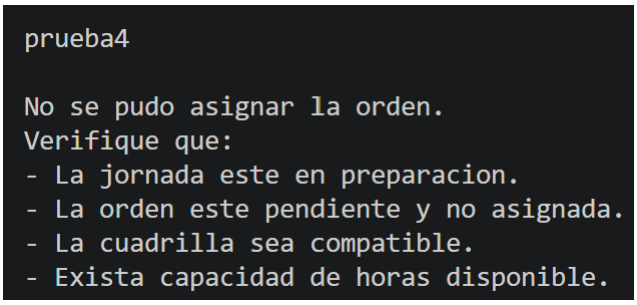


Orden efectivamente asignada



Orden de 2.5h a usar ,Efectivamente no se puedo asignar la orden por que excede H.





Prueba 12 – Asignación con capacidad exacta

- Datos de entrada: cuadrilla con capacidad de 5 horas; asignar órdenes de 3 y 2 horas.
- Resultado esperado: ambas órdenes se asignan correctamente y la capacidad disponible queda en 0 horas.
- Resultado obtenido: Orden registrada correctamente.
- Evidencia:

Utilizando las imágenes de la anterior prueba , solo colocamos una nueva orden esta vez que si sea de 2h exactas

Orden a utilizar

```
Orden registrada correctamente.
----------------------
Código: prueba5
Nombre del Cliente: prueba5
Dirección: prueba5
Horas Estimadas: 2.0
Zona: metropolitana
Estado: Pendiente
Visitas Realizadas: 0
Puntos a instalar: 4
----------------------
Costo del servicio: Q370.00
Dias estimados de atencion: 3
```

Se la asignamos a la anterior jornada.

```
prueba5

Orden asignada correctamente.

--------JORNADA--------
Código: jornada1
Estado: Preparacion
Código de la cuadrilla: waza
Códigos de órdenes:
- prueba1
- prueba5
----------------------
```

Prueba 13 – Cuadrilla sin certificación

- Datos de entrada: intentar asignar un mantenimiento de aire acondicionado a una cuadrilla sin certificación en refrigeración.
- Resultado esperado: el sistema rechaza la asignación por incompatibilidad.
- Resultado obtenido: No se pudo asignar la orden.
- Evidencia:

  En la anterior jornada , antes de verificar la prueba, por error hice esta prueba y bueno estos fueron los resultados.

  Orden usada

```
Orden registrada correctamente.
--------------------
Código: prueba2
Nombre del Cliente: prueba2
Dirección: prueba2
Horas Estimadas: 2.0
Zona: metropolitana
Estado: Pendiente
Visitas Realizadas: 0
Capacidad BTU: 25000
--------------------
```

  Cuadrilla anteriormente hecha

```
----------------------
Código: waza
Capacidad de horas: 5.0
Certificada en refrigeración: false
----------------------
```

Resultado

```
prueba2

No se pudo asignar la orden.
Verifique que:
- La jornada este en preparacion.
- La orden este pendiente y no asignada.
- La cuadrilla sea compatible.
- Exista capacidad de horas disponible.
```

Prueba 14 – Orden asignada a otra jornada

- Datos de entrada: asignar una orden a J001 y después intentar asignarla a J002.
- Resultado esperado: el sistema rechaza la segunda asignación porque la orden ya pertenece a una jornada.
- Resultado obtenido: No se pudo asignar la orden.
- Evidencia:

Comprobamos con esta orden anteriormente hecha y asignada a jornada 1

```
Orden registrada correctamente.
----------------------
Código: prueba5
Nombre del Cliente: prueba5
Dirección: prueba5
Horas Estimadas: 2.0
Zona: metropolitana
Estado: Pendiente
Visitas Realizadas: 0
Puntos a instalar: 4
----------------------
Costo del servicio: Q370.00
Dias estimados de atencion: 3
```

Cuadrilla a utilizar

```
----------------------
Código: cuadrilla2
Capacidad de horas: 5.0
Certificada en refrigeración: true
----------------------
```

Jornada a utilizar

```
--------JORNADA--------
Código: jornada2
Estado: Preparacion
Código de la cuadrilla: cuadrilla2
Códigos de órdenes:
----------------------
```

Resultado de implementar prueba 5.

```
prueba5

No se pudo asignar la orden.
Verifique que:
- La jornada este en preparacion.
- La orden este pendiente y no asignada.
- La cuadrilla sea compatible.
- Exista capacidad de horas disponible.
```

Prueba 15 – Inicio de jornada vacía

- Datos de entrada: crear una jornada sin órdenes asignadas e intentar iniciarla.
- Resultado esperado: el sistema rechaza el inicio y la jornada permanece en estado Preparacion.
- Resultado obtenido: No se pudo iniciar la jornada. Verifique que tenga ordenes asignadas y pendientes.
- Evidencia:

Cuadrilla y jornada a utilizar

```
--------JORNADA--------
Código: jornada3
Estado: Preparacion
Código de la cuadrilla: cuadrilla3
Códigos de órdenes:
----------------------
```

```
cuadrilla3

----------------------
Código: cuadrilla3
Capacidad de horas: 6.0
Certificada en refrigeración: true
----------------------
```

Resultado de iniciar la jornada:

```
jornada3

No se pudo iniciar la jornada.
Verifique que tenga ordenes asignadas y pendientes.
```

Prueba 16 – Entradas y códigos inválidos

- Datos de entrada: ingresar un código repetido, un código vacío, letras en un campo numérico y un número negativo.
- Resultado esperado: el sistema muestra mensajes de error, rechaza los datos inválidos y permite continuar utilizando el menú.
- Resultado obtenido: El sistema lanza varios errores y vuelve a pedir la entrada. Solamente regresa al menú cuando ya existe un código con esa orden.
- Evidencia:

  Esta prueba se realizo con registrar una orden

  Codigo repetido

```
Ingrese el codigo de la orden:

prueba1
Error: ya existe una orden con ese codigo.
```

  código Vacio

```
Ingrese el codigo de la orden:


Error: la entrada no puede estar vacía.
```

  Letras en un campo numerico

```
Ingrese las horas estimadas:
s
Error: debe ingresar un numero decimal valido.
```

  Numero negativo

```
Ingrese las horas estimadas:
-5
Error: el numero debe ser finito y no negativo.
```

Prueba 17 – Diagnóstico con tres visitas fallidas

- Datos de entrada: asignar un diagnóstico a tres jornadas sucesivas y registrar una visita fallida en cada una.
- Resultado esperado: después de las primeras dos visitas, la orden vuelve a Pendiente. Después de la tercera, pasa a Cancelada y no puede reasignarse.
- Resultado obtenido: Orden D001: Cancelada.

  Visitas realizadas: 3.

  Jornada J003: Cerrada.

  Cobro adicional acumulado: Q80,00.
- Evidencia:

  Volvemos a iniciar el programa para que no hayan tantas pruebas.

  Cuadrilla utilizada

```
----------------------
Código: cuadrilla 1
Capacidad de horas: 5.0
Certificada en refrigeración: true
----------------------
```

  Jornada 1 utilizada

```
--------JORNADA--------
Código: jornada 1
Estado: Preparacion
Código de la cuadrilla: cuadrilla 1
Códigos de órdenes:
- D1
----------------------
```

  Orden utilizada

```
Orden registrada correctamente.
----------------------
Código: D1
Nombre del Cliente: Jose
Dirección: Rios N
Horas Estimadas: 1.5
Zona: Metropolitana
Estado: Pendiente
Visitas Realizadas: 0
----------------------
Costo del servicio: Q150.00
Dias estimados de atencion: 1
```

  Iniciamos la jornada

```
Jornada iniciada correctamente.

--------JORNADA--------
Código: jornada 1
Estado: En curso
Código de la cuadrilla: cuadrilla 1
Códigos de órdenes:
- D1
----------------------
```

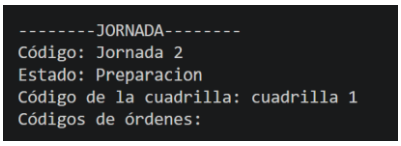
  Registramos la visita y se cierra jornada 1

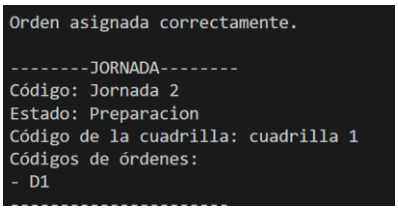
```
Resultado registrado correctamente.

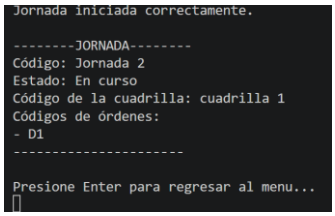
Estado actual de la orden: Pendiente
Visitas realizadas: 1

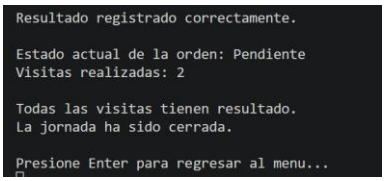
Todas las visitas tienen resultado.
La jornada ha sido cerrada.
```

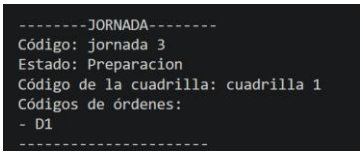
Creamos jornada 2 y repetimos este proceso hasta hacer 3 jornadas

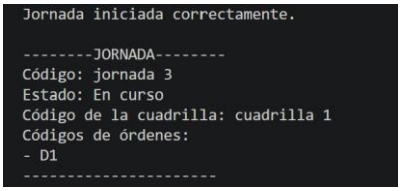












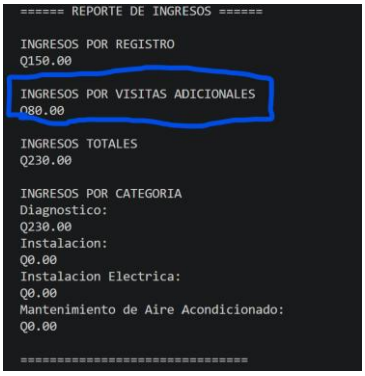
Se cancela la orden del servicio, se cierra la jornada, se realizaron 3 visitas

```
Resultado registrado correctamente.

Estado actual de la orden: Cancelada
Visitas realizadas: 3

Todas las visitas tienen resultado.
La jornada ha sido cerrada.
```

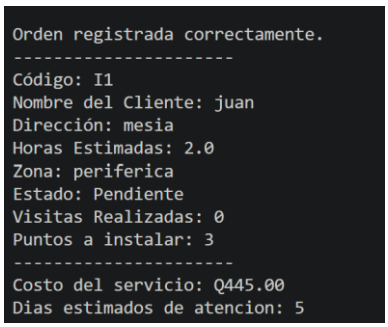
Un cobro de 80 por visitas adicionales.

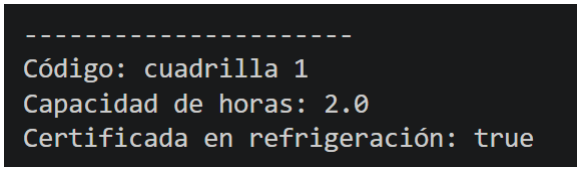


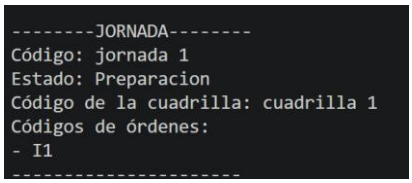
Prueba 18 – Instalación con tres visitas fallidas

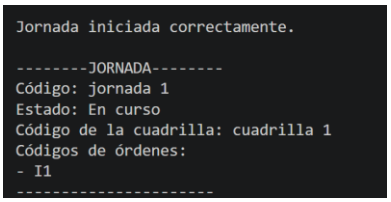
- Datos de entrada: registrar tres visitas fallidas para una instalación normal, utilizando jornadas sucesivas.
- Resultado esperado: la orden queda Cancelada después de la tercera visita.
- Resultado obtenido: Similar al anterior, solo varia en los diferentes ingresos.
- Evidencia:

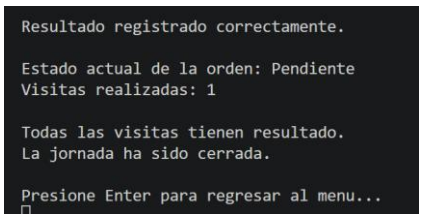
  Se hace el mismo procedimiento pero ahora en vez con una orden de instalación

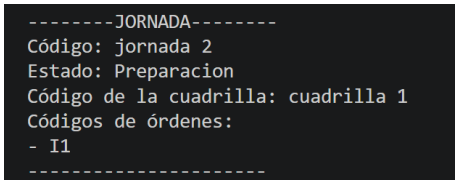


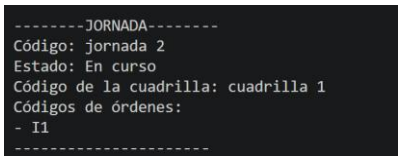


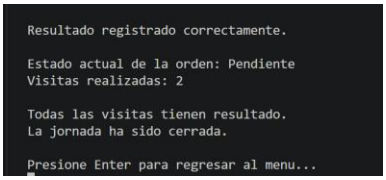












```
Orden asignada correctamente.

--------JORNADA--------
Código: jornada 3
Estado: Preparacion
Código de la cuadrilla: cuadrilla 1
Códigos de órdenes:
- I1
----------------------
```

```
Jornada iniciada correctamente.

--------JORNADA--------
Código: jornada 3
Estado: En curso
Código de la cuadrilla: cuadrilla 1
Códigos de órdenes:
- I1
----------------------
```

```
Resultado registrado correctamente.

Estado actual de la orden: Cancelada
Visitas realizadas: 3

Todas las visitas tienen resultado.
La jornada ha sido cerrada.

Presione Enter para regresar al menu...
```

```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q445.00

INGRESOS POR VISITAS ADICIONALES
Q80.00

INGRESOS TOTALES
Q525.00

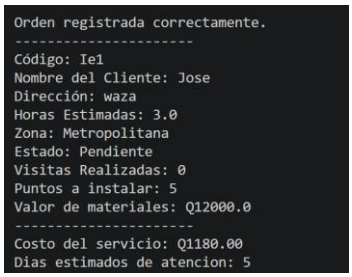
INGRESOS POR CATEGORIA
Diagnostico:
Q0.00
Instalacion:
Q525.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

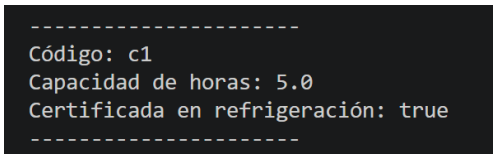
================================
```

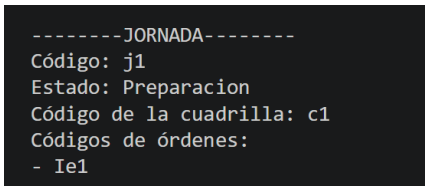
Prueba 19 – Instalación eléctrica con dos visitas fallidas

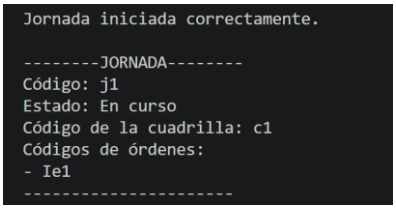
- Datos de entrada: registrar dos visitas fallidas para una instalación eléctrica certificada.
- Resultado esperado: la orden queda Cancelada después de la segunda visita y no puede asignarse nuevamente.
- Resultado obtenido: Similar al anterior, solo varia en los diferentes ingresos y se cancela a las dos visitas.
- Evidencia:

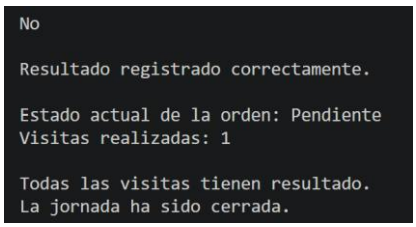
  Se hace el mismo procedimiento, pero ahora en vez con una orden de instalación

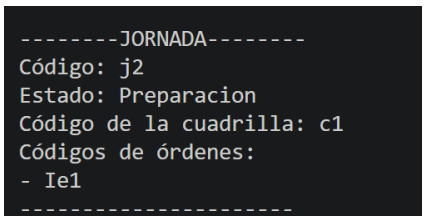


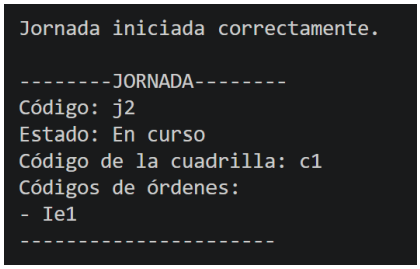


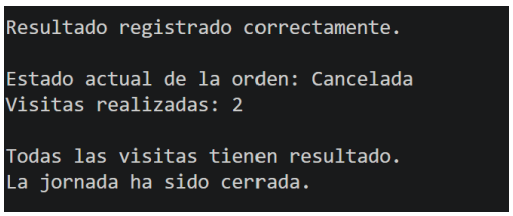












```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q1180.00

INGRESOS POR VISITAS ADICIONALES
Q40.00

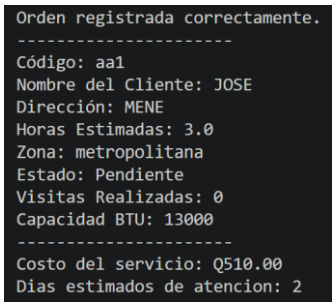
INGRESOS TOTALES
Q1220.00

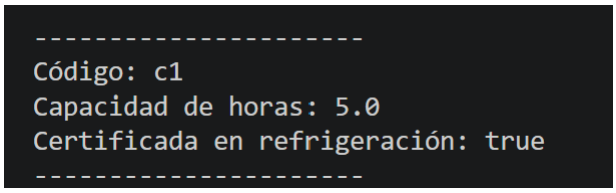
INGRESOS POR CATEGORIA
Diagnostico:
Q0.00
Instalacion:
Q0.00
Instalacion Electrica:
Q1220.00
Mantenimiento de Aire Acondicionado:
Q0.00

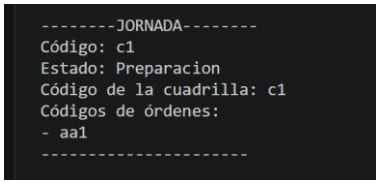
==============================
```

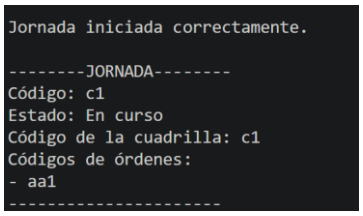
Prueba 20 – Mantenimiento con una visita fallida

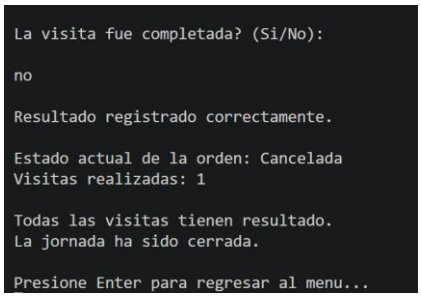
- Datos de entrada: registrar una visita fallida para un mantenimiento de aire acondicionado.
- Resultado esperado: la orden queda Cancelada inmediatamente porque solo permite una visita.
- Resultado obtenido: Similar al anterior, solo varia en los diferentes ingresos y se cancela a la primera visita.
- Evidencia:
- Se hace el mismo procedimiento pero ahora en vez con una orden de instalación

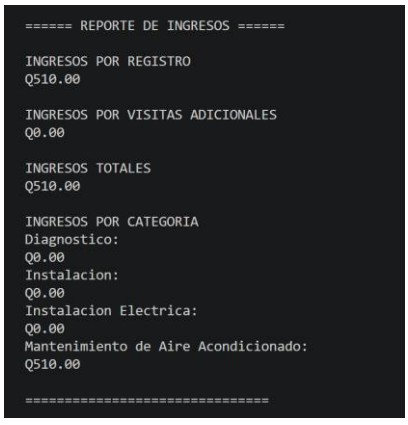












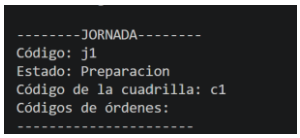
Prueba 21 – Cierre de jornada y reasignación

- Datos de entrada: crear una jornada con dos órdenes, iniciarla y registrar una visita completada y otra fallida.
- Resultado esperado: la jornada permanece En curso hasta registrar ambos resultados. Después queda Cerrada, libera la cuadrilla y permite reasignar la orden que quedó Pendiente.
- Resultado obtenido: Jornada 1 se cierra , I1 esta completado, I2 y cuadrilla se pueden asignar a J2
- Evidencia:

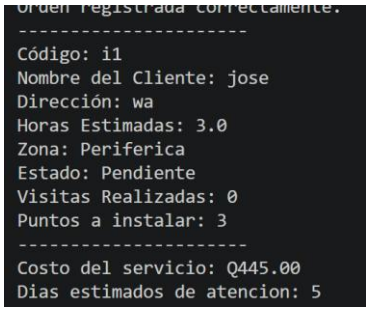
Cuadrilla utilizada



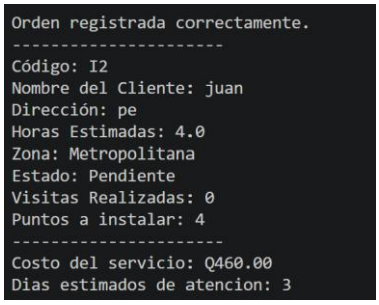
Jornada utilizada



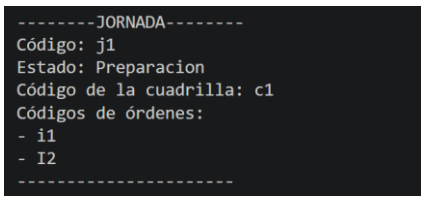
Orden utilizada 1



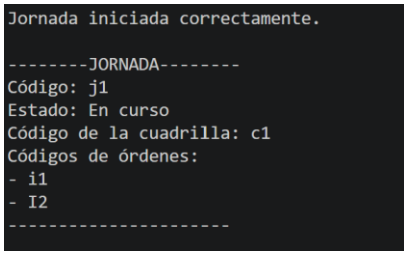
Orden utilizada 2



Asignacion



Se inicia la jornada



Se registra el resultado de la visita Sigue en curso auque este la orden i1 este completada

```
Ingrese el codigo de la jornada:

j1

Ingrese el codigo de la orden:

i1

La visita fue completada? (Si/No):

Si

Resultado registrado correctamente.

Estado actual de la orden: Completada
Visitas realizadas: 1
```

```
JORNADAS EN CURSO

--------JORNADA--------
Código: j1
Estado: En curso
Código de la cuadrilla: c1
Códigos de órdenes:
- i1
- I2
-----------------------
Horas asignadas: 7.0
Capacidad disponible: 1.0
```

Se registra la segunda visita y tenemos:

```
I2

La visita fue completada? (Si/No):

No

Resultado registrado correctamente.

Estado actual de la orden: Pendiente
Visitas realizadas: 1

Todas las visitas tienen resultado.
La jornada ha sido cerrada.

Presione Enter para regresar al menu...
```

Por ultimo se comprueba que tanto la cuadrilla como I2 puedan asignarse a otra jornada ya que su visita no se completo

```
Orden asignada correctamente.

--------JORNADA--------
Código: J2
Estado: Preparacion
Código de la cuadrilla: c1
Códigos de órdenes:
- I2
-----------------------
```

Prueba 22 – Cotización sin modificación de ingresos

- Datos de entrada: consultar el reporte de ingresos, cotizar una instalación y consultar nuevamente el reporte.
- Resultado esperado: los ingresos permanecen iguales porque la cotización no registra ni cobra una orden.
- Resultado obtenido: El reporte de ingresos no cambia absolutamente nada
- Evidencia:

  Reporte de ingresos inicial

```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q0.00

INGRESOS POR VISITAS ADICIONALES
Q0.00

INGRESOS TOTALES
Q0.00

INGRESOS POR CATEGORIA
Diagnostico:
Q0.00
Instalacion:
Q0.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

==============================
```

Se hace una cotizacion

```
====== COTIZAR SERVICIO ======
1. Diagnostico
2. Instalacion
3. Instalacion Electrica
4. Mantenimiento de Aire Acondicionado
5. Regresar al menu principal

Seleccione una opcion:
1

Ingrese el codigo de la orden:

W

Ingrese el nombre del cliente:

d

Ingrese la direccion:

De

Ingrese las horas estimadas:
1.5

Ingrese la zona (Metropolitana/Periferica):

metropolitana

====== RESULTADO DE COTIZACION ======
Costo del servicio: Q150.00
Dias estimados de atencion: 1
```

Se consulta nuevamente el reporte

```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q0.00

INGRESOS POR VISITAS ADICIONALES
Q0.00

INGRESOS TOTALES
Q0.00

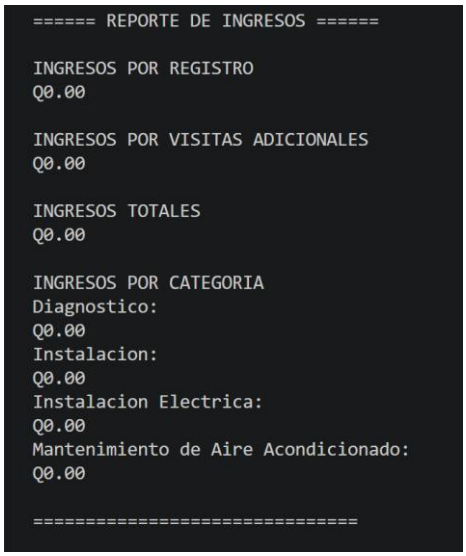
INGRESOS POR CATEGORIA
Diagnostico:
Q0.00
Instalacion:
Q0.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

================================
```

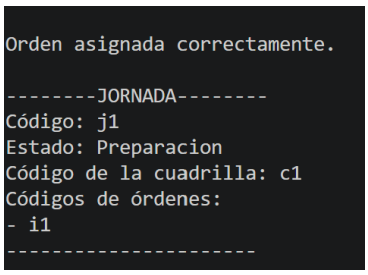
Prueba 23 – Retiro de orden sin modificación de ingresos

- Datos de entrada: consultar ingresos, asignar una orden pendiente a una jornada en preparación, retirarla y consultar nuevamente los ingresos.
- Resultado esperado: la orden queda Pendiente y sin asignación, mientras que los ingresos permanecen sin cambios.
- Resultado obtenido: La orden permanece en estado Pendiente y deja de estar asignada a la jornada. Los ingresos por registro permanecen en Q150,00, los ingresos por visitas adicionales en Q0,00 y los ingresos totales en Q150,00.
- Evidencia:

  Se consultan los ingresos



  Se asigna una orden con estos a esta jornada



Se retira la orden;

```
Ingrese el codigo de la orden que desea retirar:

i1

Orden retirada correctamente.

--------JORNADA--------
Código: j1
Estado: Preparacion
Código de la cuadrilla: c1
Códigos de órdenes:
-----------------------
```

Se comprueban el reporte de ingresos , Se registra el ingreso por registro ya que se creo la orden en zona metropolitana , pero por quitar una orden a jornada no se registro nada.

```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q150.00

INGRESOS POR VISITAS ADICIONALES
Q0.00

INGRESOS TOTALES
Q150.00

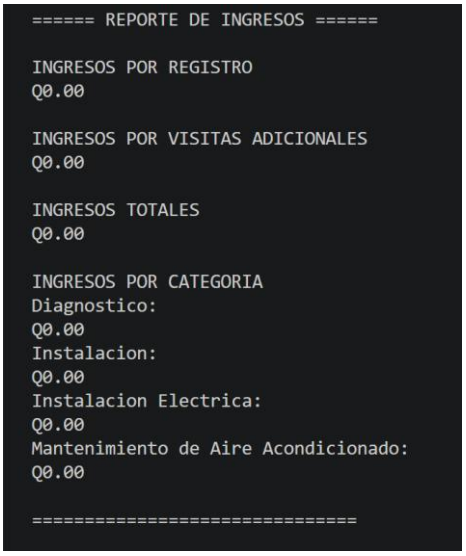
INGRESOS POR CATEGORIA
Diagnostico:
Q150.00
Instalacion:
Q0.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

================================
```

Prueba 24 – Cobro de Q40 y operaciones rechazadas

- Datos de entrada: registrar una primera visita fallida, cerrar la jornada, reasignar la orden a una nueva jornada e iniciarla. Después intentar iniciar nuevamente esa jornada y registrar dos veces el resultado de la misma orden.
- Resultado esperado: la segunda visita genera exactamente Q40,00 adicionales al iniciar la nueva jornada. Los intentos rechazados no generan nuevos cobros ni modifican los estados.
- Resultado obtenido: Se ha comprobado todo lo esperado.
- Evidencia:

Primero se realiza un reporte de ingresos para confirmar todo reiniciado.



Orden utilizada

```
----------------------
Código: d1
Nombre del Cliente: e
Dirección: e
Horas Estimadas: 1.5
Zona: Periferica
Estado: Pendiente
Visitas Realizadas: 0
----------------------
Costo del servicio: Q225.00
Dias estimados de atencion: 2
```

Cuadrilla utilizada

```
---------------------
Código: c1
Capacidad de horas: 4.0
Certificada en refrigeración: true
---------------------
```

Jornada a utilizar

```
--------JORNADA--------
Código: J1
Estado: Preparacion
Código de la cuadrilla: c1
Códigos de órdenes:
- d1
----------------------
```

Inicia la jornada

```
Jornada iniciada correctamente.

--------JORNADA--------
Código: J1
Estado: En curso
Código de la cuadrilla: c1
Códigos de órdenes:
- d1
----------------------
```

Se registra la visita fallida

```
La visita fue completada? (Si/No):

No

Resultado registrado correctamente.

Estado actual de la orden: Pendiente
Visitas realizadas: 1

Todas las visitas tienen resultado.
La jornada ha sido cerrada.

Presione Enter para regresar al menu...
```

Segunda jornada

```
Orden asignada correctamente.

--------JORNADA--------
Código: j2
Estado: Preparacion
Código de la cuadrilla: c1
Códigos de órdenes:
- d1
----------------------
```

Se inicia la segunda jornada

```
Jornada iniciada correctamente.

--------JORNADA--------
Código: j2
Estado: En curso
Código de la cuadrilla: c1
Códigos de órdenes:
- d1
-----------------------
```

Se ve que realiza el cobro de 40 por la visita.

```
======  REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q225.00

INGRESOS POR VISITAS ADICIONALES
Q40.00

INGRESOS TOTALES
Q265.00

INGRESOS POR CATEGORIA
Diagnostico:
Q265.00
Instalacion:
Q0.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

==============================


Presione Enter para regresar al menu...
```

Se inicia nuevamente la segunda jornada y marca un error ya que esta en curso.

```
Ingrese el codigo de la jornada:

j2

Error: la jornada no esta en preparacion.
```

Se registra la segunda visita

```
La visita fue completada? (Si/No):

si

Resultado registrado correctamente.

Estado actual de la orden: Completada
Visitas realizadas: 2

Todas las visitas tienen resultado.
La jornada ha sido cerrada.

Presione Enter para regresar al menu...
```

Se intenta nuevamente que la se registro una visita a la jornada pero tira error ya que esta cerrada

```
Ingrese el codigo de la jornada:

j2

Error: la jornada no esta en curso.

Presione Enter para regresar al menu...
```

Se verifican nuevamente los ingresos y se ve que no cambiaron

```
====== REPORTE DE INGRESOS ======

INGRESOS POR REGISTRO
Q225.00

INGRESOS POR VISITAS ADICIONALES
Q40.00

INGRESOS TOTALES
Q265.00

INGRESOS POR CATEGORIA
Diagnostico:
Q265.00
Instalacion:
Q0.00
Instalacion Electrica:
Q0.00
Mantenimiento de Aire Acondicionado:
Q0.00

==============================
```